

Tree :-

Hierarchical Data Structure



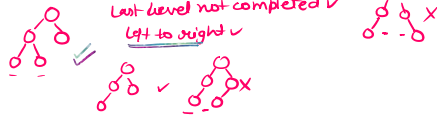
Binary tree: at max 2 children

child
parent
siblings
edge → two nodes connected by a line

Types of Binary Tree

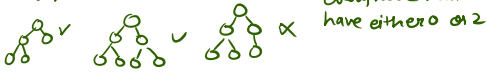


Complete BT



Last level not completed ✓
Left to right ✓

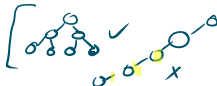
Full binary T / Strict BT



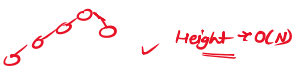
every node must have either 0 or 2

Perfect BT → all nodes are full

Height Balanced BT → $O(\log N)$



Skewed BT



Ordered BT → every node have same property → BST

Properties

① Total no. of nodes in a perfect Binary Tree

$$2^{(h+1)} - 1$$

Example: $2^{(1+1)} - 1 = 4 - 1 = 3$

② Total no. of leaf in PBT 2^h

Implementation

②

- ① Linked
- ② Sequential → array

Linked

class BinaryTree {

class Node {

int value;

Node left;

Node right;

public Node(int value) {

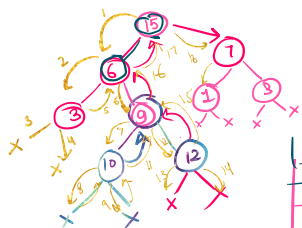
this.value = value;

this.left = null;

this.right = null;

} }

}



12
10
9
5
6
15

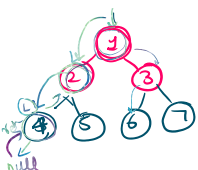
root = newNode(value);
node.left = newNode(value)

Tree Traversal

①

InOrder

Left Root Right



② PreOrder

Root Left Right

③ PostOrder

Left Right Root

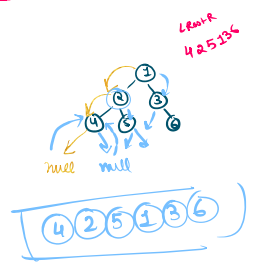
Linker
... 5136

4 5 6 7
null

Root Left Right

(4 2 5 1 6 3 7)

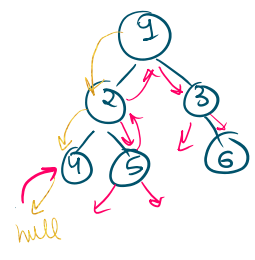
```
public static void inOrder(Node root){
    if (root == null){
        return;
    }
    inOrder(root.left);
    System.out.print(root.data + " ");
    inOrder(root.right);
}
```



③ Preorder

Root Left Right

```
public static void Preorder(Node root){
    if (root == null){
        return;
    }
    System.out.print(root.data + " ");
    Preorder(root.left);
    Preorder(root.right);
}
```

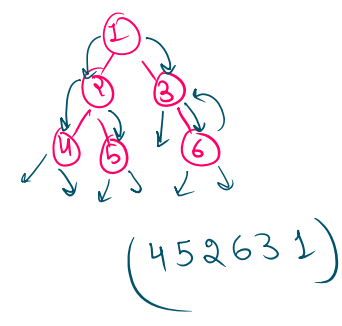


(1 2 4 5 3 6)

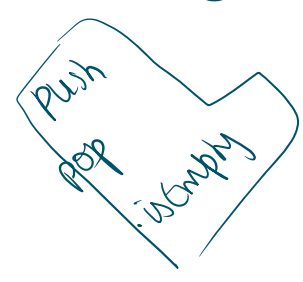
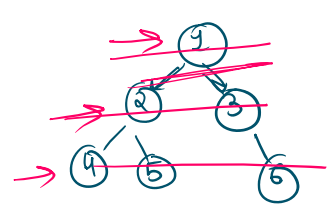
stack
dfs

③ Left Right Root

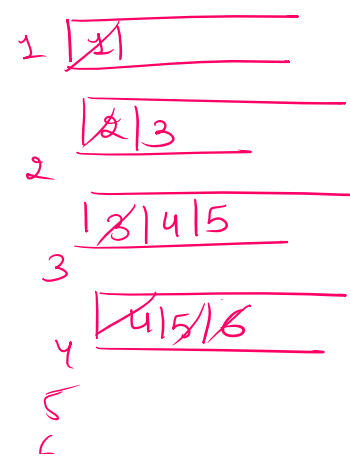
```
public static void PostOrder(Node root){
    if (root == null){
        return;
    }
    PostOrder(root.left);
    PostOrder(root.right);
    System.out.print(root.value + " ");
}
```



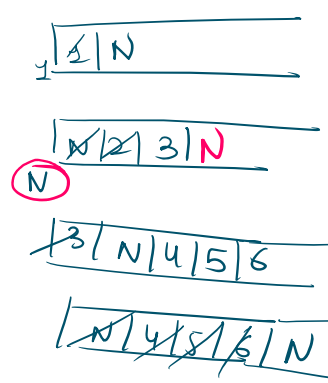
bfs → queue → Level Order Traversal



1 2 3 4 5 6



1
2 3
4 5 6



1
2 3
4 5 6

—
—

```
public static void reverseOrder (Node root) {
```

```
    if (root == null) {
```

```
        return;
```

```
    }
```

```
    Queue <Node> q = new LinkedList<>();
```

```
    q.add(root);
```

```
    q.add(null);
```

```
    while (!q.isEmpty()) {
```

```
        Node cur = q.remove();
```

```
        if (cur == null) {
```

```
            sysopln ();
```

```
            if (q.isEmpty()) {
```

```
                break;
```

```
            } else {
```

```
                q.add(null);
```

```
            }
```

```
        } else {
```

```
            sysop (cur.data + " ");
```

```
            if (cur.left != null) {
```

```
                q.add(cur.left);
```

```
            }
```

```
            if (cur.right != null) {
```

```
                q.add(cur.right);
```

```
            } } }
```

Search

